TestMate AI

Mobile Game QA & Performance Intelligence

USER MANUAL · Version 1.0

for Windows 10/11 and macOS (Apple Silicon + Intel)

Step-by-step setup, full feature reference, and troubleshooting.

Prepared for the QA team

Table of contents

1. Welcome to TestMate AI

1.1 What this tool does

TestMate AI is a desktop application that turns your laptop into a complete QA station for Android mobile games. Plug a phone into your computer, drop in an APK, press **Start Test**, and TestMate captures everything that matters: real-time FPS, CPU and memory pressure, SDK initialisation, analytics events, in-app purchase signals, crash and ANR detection, and a full security audit of the manifest.

When you stop the session, TestMate produces a single QA report with a verdict (Pass / Pass with warnings / Fail) and a list of every check that ran. The same data feeds into a deterministic **Device Prediction** engine that estimates how the build will run on flagship, mid-range, and budget hardware you don't physically own.

1.2 Who this manual is for

Two audiences:

- **QA testers** — install the app, plug in a device, run the standard test flow, and export reports. You can skim chapter 8.
- **Developers and power users** — everything above, plus the ADB tips and advanced controls in chapter 8 (wireless ADB, multiple devices, force-stop).

> **Note** TestMate AI v1.0 is currently **Android-only**. iOS device prediction is included in the Device Prediction tab as a forecast, but iOS device sessions are not yet supported in v1.

2. System requirements

TestMate AI ships everything it needs — including the Android Debug Bridge (ADB) — in the installer. You do not need to install Android Studio or any developer tools on the host machine.

Component	Requirement
Operating system	macOS 11 Big Sur or newer (Apple Silicon or Intel) Windows 10 or 11 (64-bit)
Disk space	≈ 250 MB after install
RAM	4 GB minimum, 8 GB recommended
USB port	USB-A or USB-C (with appropriate cable for your device)
Android device	Android 7.0 (Nougat) or newer with USB debugging available
Internet (optional)	Only needed for update checks

3. Installation — macOS

Pick the build that matches your Mac's processor. If you're unsure, click the Apple menu → **About This Mac**; if it says *Apple M1 / M2 / M3 / M4*, you need the arm64 build, otherwise pick the Intel build.

Mac type	File	Size
Apple Silicon (M-series)	TestMate AI-1.0.0-arm64.dmg	≈ 103 MB
Intel	TestMate AI-1.0.0.dmg	≈ 108 MB

Install steps

1. Double-click the downloaded **.dmg** file. A new window opens showing the TestMate AI icon and a shortcut to the Applications folder.
2. Drag the **TestMate AI** icon onto the **Applications** folder.
3. Wait a few seconds for the copy to finish, then eject the .dmg from Finder.
4. Open **Applications** and find **TestMate AI**.

> **Important** Because v1 is not yet notarised by Apple, the first launch will show a warning that the app is from an unidentified developer or is “damaged”. This is expected. **Right-click** (or Control-click) the app icon → **Open** → confirm. After this one-time approval, double-click works normally.

If right-click → Open still refuses, open Terminal and run:

```
xattr -cr "/Applications/TestMate AI.app"
```

This clears the macOS quarantine attribute. You only need to do it once.

4. Installation — Windows 10 / 11

File	Size
TestMate AI Setup 1.0.0.exe	≈ 78 MB

Install steps

1. Double-click **TestMate AI Setup 1.0.0.exe**.
2. Windows SmartScreen may show *"Windows protected your PC"*. Click **More info**, then **Run anyway**. (This warning appears because v1 is unsigned; future versions will be Authenticode-signed.)
3. If User Account Control prompts you, click **Yes**.
4. Choose an install location (the default **Program Files\TestMate AI** is fine), then click **Install**.
5. When install finishes, tick **Launch TestMate AI** and click **Finish**. The app also appears in the Start menu and (optionally) on the Desktop.

> **Driver note** Some Android phones need an OEM USB driver on Windows the first time you connect them (Samsung, Xiaomi, OPPO, Vivo). If your device shows up as **OFFLINE** in TestMate AI even after enabling USB debugging, search for "[your phone brand] USB driver Windows" and install the OEM-provided driver. Pixel and Nothing devices work out of the box.

5. First-time setup

5.1 Enable USB debugging on your Android device

USB debugging is the channel TestMate AI uses to read telemetry from the phone. It is hidden behind Developer Options on every Android device.

1. On the device, open **Settings** → **About phone**.
2. Find the **Build number** entry and tap it **seven times**. A toast says *"You are now a developer!"*
3. Go back to **Settings** → **System** → **Developer options**. (On some skins it's directly under Settings.)
4. Toggle **USB debugging** ON. Confirm the warning dialog.
5. *(Optional but recommended)* turn on **Stay awake** so the screen doesn't lock during long test sessions.

5.2 Connect the device to TestMate AI

1. Plug the phone into your laptop with a **data-capable** USB cable. (Some cheap cables only carry power and won't work — if you see no device, try a different cable first.)
2. On the device, the prompt *"Allow USB debugging?"* appears with your computer's RSA key fingerprint. Tick **Always allow from this computer** and tap **Allow**.
3. Open TestMate AI and click the **Test Session** tab in the sidebar.
4. Look at the device status bar at the top. It should show your phone model and **CONNECTED**.

> **Troubleshooting tip** If the device shows OFFLINE or doesn't appear: unplug and re-plug the cable, then in the phone's Developer Options tap **Revoke USB debugging authorizations** and re-plug. Re-accept the prompt.

5.3 Create your first project

A **project** in TestMate AI groups all sessions and reports for a single game. Most studios make one project per title.

1. In the left sidebar, click the **+** icon next to *Projects*.
2. Type a project name (e.g. *MyGame*) and press **Enter**. The project becomes active and is highlighted in the sidebar.
3. All sessions you record from now on are saved under this project, until you switch or create another.

6. Tour of the interface

The TestMate AI window is split into a **left sidebar** for projects and navigation, and a **main panel** that swaps content based on which tab you select. The dark theme is the default, with the Linear/Vercel-inspired blue accent palette; a light theme is available from the footer settings.

Sidebar tab	What it does
Test Session	The control room. Drop an APK, start/stop the live test, watch real-time FPS and CPU, see automated checks tick green.
Runtime Intel	Live-detected SDKs (ad networks, analytics, attribution) compared against a reference catalogue.
Runtime Events	Live timeline of analytics events (GameAnalytics, Firebase, Facebook, IAP, AppsFlyer, Adjust, Unity lifecycle) with category filters.
IAP Validation	End-to-end test of an in-app purchase: triggers Billing flow, validates the transaction signal, measures latency.
Build Regression	Compare two builds side-by-side. Highlights perf, SDK and security deltas.
QA Checklist	Two modes — **Automated** (auto-ticked from session data) and **Manual** (20 sections, 111 items, exportable as JSON).
Static Analysis	Manifest, permissions, target SDK, cleartext traffic, native architectures, size breakdown — all from the APK alone, no device needed.
Device Prediction	Forecasts FPS / thermal headroom across flagship, mid-range and budget devices (Android catalogue, plus iOS forecast view).
History	Every saved session, with verdict, key metrics, full report JSON export.

6.1 Feature accuracy & status at a glance

Each feature in TestMate AI v1.0 has been calibrated against real builds. The table below summarises the confidence you should put in each feature's output, and flags the two features that are still in active development for v1.x.

Feature	Accuracy	Status
Test Session (live telemetry)	≈ 98 %	Stable
Runtime Intel (SDK detection)	≈ 95 %	Stable
Runtime Events	≈ 100 % (logcat)	Stable
IAP Validation	Partial	In development*
Build Regression	Partial	In development*
QA Checklist — Automated	≈ 95 %	Stable

Feature	Accuracy	Status
QA Checklist — Manual	100 % (human)	Stable
Static Analysis (APK manifest, perms)	≈ 99 %	Stable
Device Prediction (Android catalogue)	≈ 88 %	Stable
History (session storage & export)	100 %	Stable

** Features marked “In development” are included in v1.0 as a preview. Their results may be incomplete or incorrect — please double-check manually and report any inconsistencies before relying on them for release decisions.*

7. Feature guide

Each subsection below covers one tab in the order it appears in the sidebar. The format is the same: **What it is** → **How to use it** → **What to look for**.

7.1 Test Session

Accuracy: ≈ 98 % · **Status:** Stable

What it is

The primary workspace. While a session is running TestMate AI streams telemetry from the device — FPS, frame time, CPU per-core, memory, GPU load, network — and drives every other tab in real time.

How to use it

1. Confirm the device shows **CONNECTED** in the status bar.
2. **Drag and drop** your APK onto the dashed drop zone. (You can also click **Browse** and pick the file.) TestMate parses the APK and fills in the package name, version, target SDK, and size.
3. Click ■ **Start Test**. TestMate installs the APK on the device, launches the main activity, and begins live monitoring. The session timer starts.
4. **Play the game.** TestMate captures everything in the background — you don't need to babysit the dashboard. Watch the milestone bar fill up as automated checks tick green (Game Start, Splash, Firebase init, Ads SDK init, etc.).
5. When you've covered the scenarios you want to test, click ■ **Stop**. TestMate kills the app, generates the report, and saves the session into **History**.

What to look for

- **FPS chart** — should stay at the device's target refresh rate (60 / 90 / 120). Sustained dips below 30 FPS are a red flag.
- **CPU** — bursts above 80% on a single core for more than a few seconds usually mean a main-thread bottleneck.
- **Memory** — a steady upward slope with no plateau is a leak.
- **Automated checks** — every red ■ in the milestone strip is a finding worth investigating before you ship.

7.2 Runtime Intel

Accuracy: ≈ 95 % · **Status:** Stable

What it is

A live audit of every SDK that initialises while the game is running, organised into ad networks, analytics, attribution, crash reporting, and notification SDKs. Each detected SDK is matched against TestMate's reference catalogue so you can spot version mismatches and missing init calls.

How to use it

1. Start a Test Session as in 7.1.
2. Click **Runtime Intel** in the sidebar.
3. The two-column table fills in as SDKs initialise. The left column is what TestMate *found*; the right column is the reference list of what the SDK is *supposed* to log.
4. Anything in the reference column that never appears in the “found” column is highlighted — that SDK is bundled but not initialised.

7.3 Runtime Events

Accuracy: ≈ 100 % (logcat-based) · **Status:** Stable

What it is

A real-time, filterable timeline of every business event the game fires, parsed directly from the device's logcat stream. Supports GameAnalytics (DESIGN, PROGRESSION, BUSINESS, ERROR), Firebase Analytics named events, Facebook SDK, Google Play Billing & Unity IAP, AppsFlyer, Adjust, and Unity engine lifecycle events.

How to use it

1. Open the **Runtime Events** tab. The feed is empty until the game fires something.
2. Use the **category filters** at the top (GA, Firebase, IAP, FB, etc.) to isolate one SDK's traffic.
3. Each row shows timestamp, SDK, event name, and parameters. Detection is direct from logcat, so matches are reported with high confidence.
4. Right-click any event to copy its raw payload, or to export the visible feed as JSON.

> **Pro tip** If you see “No events” for an SDK that you know is bundled, the SDK's *Initialize()* method probably wasn't called in your game code. TestMate can only detect what the game actually attempts to send. Note that release builds with stripped logs may produce gaps — use a debug build for full coverage.

7.4 IAP Validation

Accuracy: Partial · **Status:** In development — may not give correct results

> **Heads up** **IAP Validation is still in development for v1.0.** Detection of the Billing handshake works, but receipt validation and backend acknowledgement may be incomplete or report false negatives. Treat the result as a *signal*, not a verdict, and confirm purchases manually in the Play Console before relying on this feature.

What it is

A focused workflow for validating one in-app purchase end-to-end: from the user tapping *Buy*, through Google Play Billing, to your backend signal.

How to use it

1. Start a Test Session and switch to the **IAP Validation** tab.
2. Click **Begin IAP Test**. The IAP timer starts and TestMate begins watching for Billing events.
3. In the game, navigate to your shop and complete a test purchase (use a Google Play test account so you don't get charged).
4. TestMate fills in the validation card: time-to-receipt, backend acknowledgement, and any error code. A green **VALIDATED** badge means the entire flow worked.

7.5 Build Regression

Accuracy: Partial · **Status:** In development — may not give correct results

> **Heads up** **Build Regression is still in development for v1.0.** The diff engine works for FPS / CPU / memory and basic SDK adds & removes, but edge cases (scenario mismatch, thermal-throttled baselines, partial sessions) can produce misleading deltas. Use it as a starting point for investigation, not as a go/no-go gate.

What it is

Compare two completed sessions to see exactly what changed between builds. Use it before every release to catch silent perf regressions, dropped SDKs, or new permissions.

How to use it

1. Run a session against your **baseline** build. Save it.
2. Run a second session against your **candidate** build (same project, same device, same scenario).
3. Open the **Build Regression** tab and pick the two sessions from the dropdowns.
4. TestMate produces a side-by-side diff: avg/min/max FPS deltas, CPU + memory deltas, SDK adds & removes, permission changes, and APK size diff. Anything worse than the baseline is highlighted in red.

7.6 QA Checklist

Accuracy: ≈ 95 % (Automated) · 100 % (Manual, human-driven) · **Status:** Stable

What it is

A structured pre-release checklist with two modes:

- **Automated** — items are auto-ticked from the active session (Firebase init, Ads SDK detected, AppsFlyer present, no crashes, target SDK compliant, etc.).
- **Manual** — 20 sections covering 111 items (UX, sound, accessibility, store listing assets, legal, etc.) that a human tester walks through.

How to use it

1. Switch the toggle at the top to **Automated** or **Manual**.
2. In manual mode, click each item to mark it Pass / Fail / N/A and add a comment.
3. When you're done, click **Export**. You get a JSON file with the run summary, ready to attach to a release ticket.

7.7 Static Analysis

Accuracy: ≈ 99 % · **Status:** Stable

What it is

Everything you can learn from the APK without touching a device: package metadata, permissions list, target / min SDK, native architectures, signing certificate, manifest vulnerabilities (cleartext traffic, exported components, debuggable flag), and a per-folder size breakdown.

How to use it

1. Drop an APK in the **Test Session** tab. (Static analysis runs automatically — you don't need to start a live session.)
2. Open the **Static Analysis** tab to see the full report.
3. Items flagged **HIGH** severity in red should be fixed before release; **MEDIUM** items are advisories; **LOW** are informational.

7.8 Device Prediction

Accuracy: ≈ 88 % (Android catalogue) · **Status:** Stable

What it is

TestMate's deterministic scaling engine takes the live telemetry from your test device and projects how the same build will perform on hardware you don't own — across flagship, mid-range, and budget tiers. Switch between an **Android** device catalogue and an **iOS** forecast view.

How to use it

1. Run a Test Session for at least **2–3 minutes** of representative gameplay so the engine has stable data to work from.
2. Open **Device Prediction**. Pick the **Android** or **iOS** tab.
3. The table lists popular devices in each tier with predicted average FPS, thermal headroom, and a colour-coded verdict (✓ Smooth / ■ Borderline / ✗ Avoid).

> **Note** Predictions are most accurate when the test device is in the middle of the range you care about. Testing only on a flagship and predicting budget devices works, but a Pixel 6a or similar mid-range device gives the best calibration.

7.9 History

Accuracy: 100 % · **Status:** Stable

What it is

Every saved session, scoped to the active project. The list shows date, device, build version, duration, verdict, and quick-glance metrics.

How to use it

1. Click any session row to open the full report on the right.
2. Use the **Export** button to save the report as JSON (good for attaching to Jira/Linear tickets) or as a shareable PDF summary.
3. Use the trash icon to delete a session you no longer need. Deletes are permanent.

8. ADB tips and advanced controls

TestMate ships its own ADB binary so you don't need Android Studio installed. The bundled binary lives at **Contents/Resources/tools/mac/adb** on Mac and **resources\tools\win\adb.exe** on Windows. It does *not* conflict with any system ADB you may already have.

- **Wireless ADB (Android 11+)** — pair the phone over Wi-Fi from Developer Options → Wireless debugging, then in TestMate's status bar click **+ Wireless** and paste the IP:port.
- **Multiple devices** — when more than one Android device is connected, the status bar lets you pick which one a session targets.
- **Force-stop a stuck session** — clicking ■ Stop sends the standard kill; if the game ignores it, the same button held for 2 seconds issues a hard *am force-stop*.
- **Debug builds give better coverage** — release builds with R8 / ProGuard strip log output, which can leave gaps in Runtime Events. Where possible, run QA passes against a debug build of the same APK.

9. Troubleshooting

Symptom	Likely cause	Fix
Device shows OFFLINE	USB debugging not authorised, or auth was revoked.	Unplug, on-device tap Revoke USB debugging authorisations, replug, accept the prompt.
Device not detected at all (Windows)	Missing OEM USB driver.	Install the OEM driver for your phone brand (Samsung, Xiaomi, OPPO, Vivo).
Mac says “TestMate AI is damaged”	macOS quarantine flag on unsigned v1 build.	Right-click → Open the first time, or run `xattr -cr "/Applications/TestMate AI.app"`
Windows SmartScreen blocks the installer	v1 is unsigned.	Click *More info* → *Run anyway*. Future versions will be signed.
FPS chart is flat at zero	App didn't launch, or an overlay is covering the game.	Check the device — is the game actually running? Stop and restart the session.
Runtime Events feed is empty	SDKs not initialised, or release build with stripped logs.	Confirm SDK init in code. For release builds, switch to a debug build to restore log coverage.
IAP test never validates	Test account not configured in Play Console, or app not signed correctly.	Use a Google Play licence-tester account; verify the upload key matches the build.
Static Analysis tab is blank	No APK loaded.	Drop an APK in the Test Session tab first.
App crashes on launch / blank window	Corrupt install or out-of-date OS.	Reinstall TestMate AI; ensure macOS 11+ or Windows 10/11 64-bit.

10. Reporting bugs and getting help

When something doesn't behave the way this manual describes, file a report so it can be fixed in the next build. The more of the following you attach, the faster it gets resolved.

- A **screenshot** of the screen at the moment the issue happened.
- The **session report JSON** — open **History**, click the affected session, then **Export**.
- If the app itself crashed: the **console log**. Open **View** → **Toggle Developer Tools** → **Console** and copy the output.
- Your **OS version** and **device model + Android version**.
- A short note describing what you were doing right before the issue appeared.

> **Thank you** TestMate AI v1.0 is a fresh release. Real testing in real studios is what turns v1 into a v2 that's faster, sharper, and catches more issues automatically. Every report and suggestion lands in the backlog.